

Documentação do Código de Algoritmo Genético para Otimização de Hiperparâmetros do LogisticRegression

1. Visão Geral

O código apresentado utiliza Algoritmos Genéticos (AG) para otimização automática de hiperparâmetros do classificador LogisticRegression da biblioteca scikit-learn. O objetivo é buscar, no espaço definido, a combinação de hiperparâmetros que maximize uma função de "fitness" construída sobre métricas avaliadas em dados de treino e teste, penalizando possíveis casos de overfitting.

2. Estrutura do Código

O código está dividido nas seguintes etapas e funções principais:

2.1 Espaço de Busca

Define os valores possíveis para cada hiperparâmetro que será otimizado:

```
espaco_busca = {  
    'model__C': [...],  
    'model__penalty': [...],  
    'model__solver': [...],  
    'model__class_weight': [...],  
    'model__fit_intercept': [...],  
    'model__tol': [...],  
    'model__l1_ratio': [...]  
}
```

Comentário: Cada hiperparâmetro listado contém um array de valores possíveis. Durante a execução do algoritmo genético, cada indivíduo será uma combinação (índices desses arrays) representando uma configuração do modelo.

Impacto Hiperparâmetro:

Hiperparâmetro	Impacto no modelo	Impacto esperado no diagnóstico de PCOS
penalty	Afeta diretamente a complexidade do modelo, a seleção de variáveis e a capacidade de generalização.	A penalização L1 ajuda a eliminar variáveis pouco relevantes, tornando o modelo mais interpretável e reduzindo o risco de overfitting. Isso tende a manter uma boa precisão sem comprometer o recall , aspecto importante para identificar pacientes com PCOS.
tol	Define o critério de convergência do algoritmo. Valores menores aumentam a precisão da otimização, mas também o tempo de treinamento.	Tem pouca influência direta sobre recall e precisão . Seu principal impacto é permitir uma convergência adequada do modelo, evitando interrupções prematuras do treinamento.
C	Controla a intensidade da regularização. Valores menores aumentam a regularização; valores maiores tornam o modelo mais flexível.	Valores maiores permitem que o modelo capture padrões mais complexos, podendo aumentar o recall ao identificar mais casos de PCOS. Entretanto, valores muito elevados podem reduzir a precisão caso ocorram mais falsos positivos.
fit_intercept	Define se o modelo aprende um termo de intercepto independente das variáveis de entrada.	Quando os dados não estão centralizados, pode melhorar o ajuste da fronteira de decisão, proporcionando pequenos ganhos em recall e precisão , dependendo das características do conjunto de dados.
class_weight	Ajusta automaticamente o peso de cada classe durante o treinamento.	Em bases desbalanceadas, como é comum em problemas médicos, a opção balanced aumenta a importância da classe positiva, elevando principalmente o recall e reduzindo falsos negativos, sem causar perdas expressivas de precisão.
solver	Define o algoritmo de otimização utilizado no treinamento, afetando velocidade, convergência e compatibilidade com diferentes regularizações.	Não altera diretamente as métricas, mas um solver adequado, como liblinear , garante uma convergência mais estável, favorecendo modelos com melhor equilíbrio entre recall e precisão .
l1_ratio	Controla o equilíbrio entre as penalizações L1 e L2 quando utilizada a regularização Elastic Net.	Não possui efeito quando a penalização escolhida é apenas L1 ou L2 . Neste trabalho permaneceu como None , pois todas as melhores soluções utilizaram penalização L1 .

A escolha dos hiperparâmetros foi direcionada para maximizar o recall sem comprometer a precisão, considerando o contexto do diagnóstico de PCOS, em que reduzir falsos negativos é uma prioridade. Além disso, buscou-se uma configuração que proporcionasse boa capacidade de generalização, convergência estável durante o treinamento e um tempo de processamento adequado, resultando em um modelo mais robusto e confiável para apoio ao diagnóstico.

2.2 Ajuste de Validade

```
def ajustar_validade(ind):
```

```
    ...
```

```
    return ind
```

Função: Garante que o indivíduo representa uma configuração válida de hiperparâmetros. Corrige combinações que não são aceitas pelo LogisticRegression, evitando erros de execução ao forçar valores validos ao indivíduo. Exemplo: para `penalty l1` só são aceitos solvers `liblinear` ou `saga`.

2.3 Geração de Indivíduos

```
def gerar_individuo():  
    ...  
    return ajustar_validade(ind)
```

Função: Gera um indivíduo (cromossomo), ou seja, escolhe aleatoriamente um índice para cada hiperparâmetro, e depois valida a configuração usando ajustar_validade.

2.4 Decodificação

```
def decodificar(ind):  
    ...  
    return params
```

Função: Converte o indivíduo (índices dos hiperparâmetros) em um dicionário de valores reais para construir o modelo LogisticRegression no scikit-learn.

2.5 Cálculo do "Fitness"

```
def calcula_fitness(recall, precision, roc_train, roc_test):  
    gap = max(0, roc_train - roc_test)  
    if gap > 0.10:  
        penalizacao = 0  
    elif gap >= 0.05:  
        penalizacao = 0.5  
    else:  
        penalizacao = 1  
    return penalizacao * (0.6 * recall + 0.2 * precision + 0.2 * roc_test)  
    return fitness
```

Função: Calcula o score final do indivíduo, ponderando recall, precisão e ROC-AUC, e penalizando situações de overfitting (gap elevado entre o AUC de treino e de teste). Se o gap for maior que 10%, o fitness é zerado, indicando alto overfitting; se ficar entre 5% e 10%, o valor do fitness é reduzido pela metade; e, caso o gap seja menor que 5%, o fitness permanece inalterado, mantendo o resultado ponderado das métricas de avaliação do modelo. O peso dos scores forma definidos principalmente pelo contexto médico, onde o recall se mostra uma das métricas de avaliação mais importantes, porém

não queremos renunciar à precisão. Então o nosso maior desafio será achar um bom recall com uma boa precision.

2.6 Avaliação do Indivíduo

```
def avaliar(individuo, X_train, y_train, X_test, y_test):
```

```
    ...
```

```
    return {...}
```

Função: Treina o modelo com os hiperparâmetros do indivíduo, avalia no conjunto de teste, calcula métricas (recall, precision, roc_auc), e retorna os resultados junto ao fitness.

2.7 Seleção por Torneio

```
def torneio(pop, k=2):
```

```
    ...
```

```
    return vencedor
```

Função: Seleciona aleatoriamente k indivíduos da população e retorna aquele com maior valor de fitness, que será utilizado como um dos pais no processo de crossover.

Neste trabalho, foi adotado $k = 2$, proporcionando uma pressão seletiva moderada. Essa escolha evita uma seleção excessivamente elitista e preserva a diversidade genética da população, o que é especialmente importante devido à elevada taxa de mutação utilizada nos experimentos, que já promove uma exploração significativa do espaço de busca. Dessa forma, busca-se equilibrar exploração e exploração das melhores soluções ao longo das gerações.

2.8 Crossover

```
def crossover(p1, p2):
```

```
    ...
```

```
    return ajustar_validade(filho)
```

Função: Para o crossover escolhemos um crossover uniforme. Para cada hiperparâmetro, existe 50% de chance de herdar o valor do primeiro pai e 50% de chance de herdar o valor do segundo. Dessa forma, o filho pode reunir características de ambos os pais e criar novas combinações de hiperparâmetros que ainda não foram testadas. Após o cruzamento, a função `ajustar_validade()` verifica se a combinação gerada é válida para a Regressão Logística e, caso necessário, faz os ajustes para garantir a compatibilidade entre os hiperparâmetros.

2.9 Mutação

```
def mutacao(...):  
    ...  
    return ajustar_validade(ind)
```

Função: Para mutação escolhemos dois cenários:

1. Mutação adaptativa: Onde a mutação começa alta nas primeiras gerações e vai decaindo ao longo do processo até a geração final ou estagnação de fitness
2. Mutação Fixa com taxa de 0.5 e 0.8, buscando uma maior exploração nas novas gerações.

A idéia é expandir bastante o nosso espaço de busca dado que já estamos aplicando o conceito de cache para evitar testes repetitivos, então optamos por mutações agressivas para aumentar o nosso poder de exploração.

2.10 Elitismo

```
def elitismo(pop, n=5):  
    ...  
    return melhores
```

Função: Mantém os 5 melhores indivíduos para as próximas gerações. Como nossa mutação já possui uma taxa alta para aumentar a exploração do espaço de busca, a ideia do elitismo é garantir que algumas das melhores soluções encontradas sejam preservadas, evitando que boas combinações de hiperparâmetros sejam perdidas durante a evolução do algoritmo.

2.11 Nova Geração

```
def nova_geracao(...):  
    ...  
    return nova
```

Função: Gera a nova população através de elitismo e reprodução (crossover/mutação), evitando duplicatas já avaliadas e garantindo que todos os indivíduos sejam válidos.

População inicial: Para população Inicial optamos por uma população inicial aleatória, onde dentro das opções de busca ele irá gerar uma população válida para o início dos nossos testes. Esta escolha também influenciou na abordagem de mutações bem agressivas para o AG.

2.12 Execução do Algoritmo Genético

```
def run_ag_experimento(...):
```

```
    ...
```

```
    return melhor_global, historico
```

Função: Realiza todo o ciclo do algoritmo genético: inicia a população, executa avaliações, seleção, cruzamentos, mutações, elitismo e monitora estagnação de fitness. Retorna o melhor indivíduo global e o histórico.

3. Comentários Adicionais/Lembretes

- **Cache:** O algoritmo mantém um cache dos indivíduos já avaliados em cada experimento, evitando repetições/desperdício computacional.
- **Diversidade:** Caso não seja possível gerar filhos novos via crossover/mutação após várias tentativas, cria-se um indivíduo aleatório novo para manter variação genética.
- **População Inicial:** Optamos por uma certa variação entre as populações dos testes dado já termos cache arquivado durante a execução dos caches anteriores.
- **Geração Máxima:** Está sendo setada em 200 por conta do nosso poder computacional. E por conta de testes que mostraram que perto desta geração já teríamos um excelente ganho.
- **Early Stop:** Definimos como Early Stop 80 gerações com estagnação de fitness dado o nosso poder computacional.

4.Resultados/Ganhos:

	Modelo	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Original - Sem AG	0.8991	0.8205	0.8889	0.8533	0.9509
1	AG Pop80 (Mut.Adaptativa)	0.9358	0.8919	0.9167	0.9041	0.9517
2	AG Pop90 (Mut.0.5)	0.9541	0.9189	0.9444	0.9315	0.9528
3	AG Pop100 (Mut.0.8)	0.9541	0.9189	0.9444	0.9315	0.9532

Comparando com o modelo original sem o uso do algoritmo genético, observamos ganhos relevantes em todos os testes realizados. O principal resultado alcançado foi o aumento do recall sem uma perda significativa de precisão, o que representa um cenário bastante positivo na construção de modelos de classificação. Dessa forma, conseguimos obter modelos mais equilibrados, assertivos e robustos em comparação ao modelo original.

5. Aplicação

O código pode ser aplicado para ajuste automático de hiperparâmetros em problemas de classificação binária, maximizando métricas específicas, e reduzindo o risco de overfitting, adaptável para outros modelos e métricas conforme necessário.